PW Earners

Promises → ES6 / 2015

ecmascript

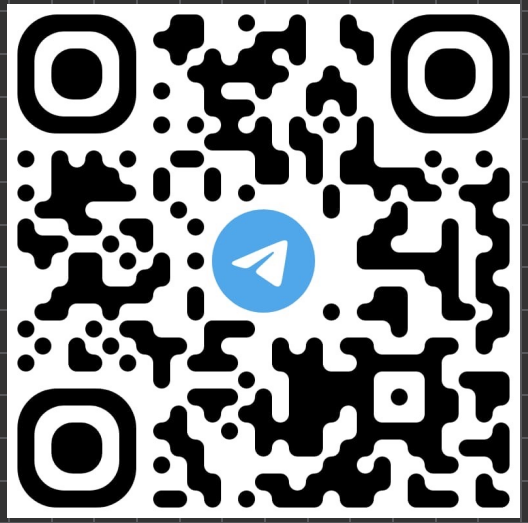
syntax

BTS

behind the scene

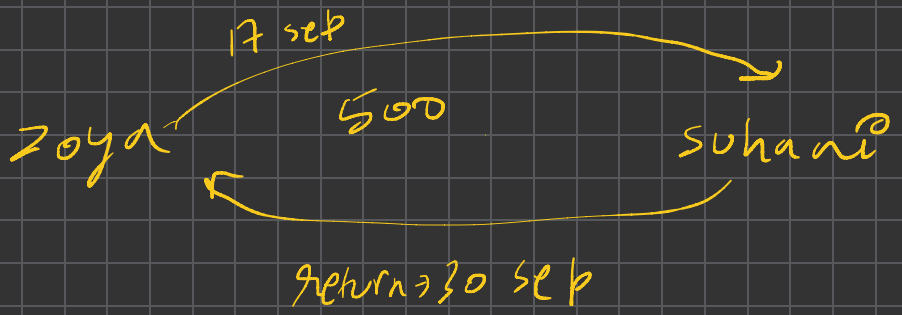
Promises → callback

Telegram



Discord





eg 18 sep – 29 sep

promise → pending

complete ✗

Reject ✗

eg 30 sep

promise → agar 500 mil gaye toh completed

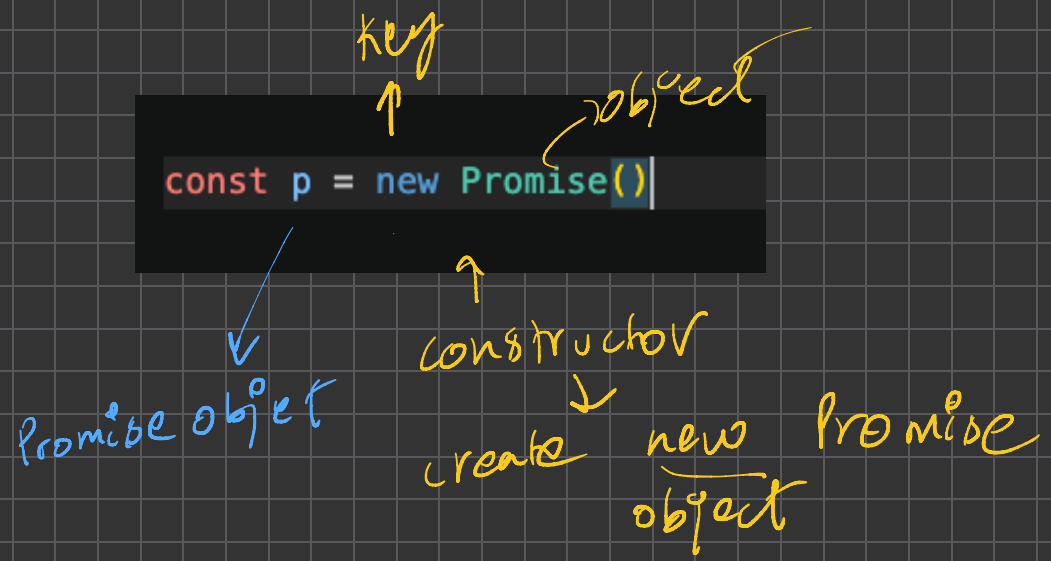
otherwise → Rejected

↓

ho promise settle ho jayega

JS promises

A promise is an object that represents the future result of an asynchronous task/operation


key
object
const p = new Promise()
Promise object
constructor
create new Promise object

new Promise( )

executor

() => {} → arrow

function () {} → anonymous

function fun1() {} → named

function (resolve , reject) {

}

resolve & reject are also functions

Promise has 3 states

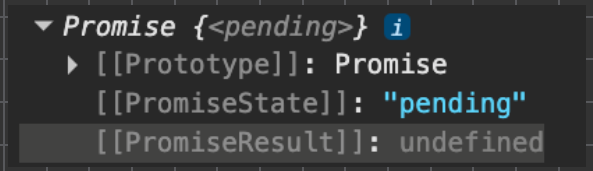
initial

fulfilled
completed
resolved

Rejected

initial state se Promise always

settled hoga



settled → either fulfilled or rejected

value | Result

initially ⇒ undefined

JS

pizza = searchPizza()

↓

JS promise karega

↓

result tumko milega

↓

rejected resolve

Once promise becomes fulfilled/rejected (settled),

it doesn't go back to pending.

above is the solution of Inversion of control

creating a Promise is synchronous.

The work inside it can be asynchronous

How to consume Promises

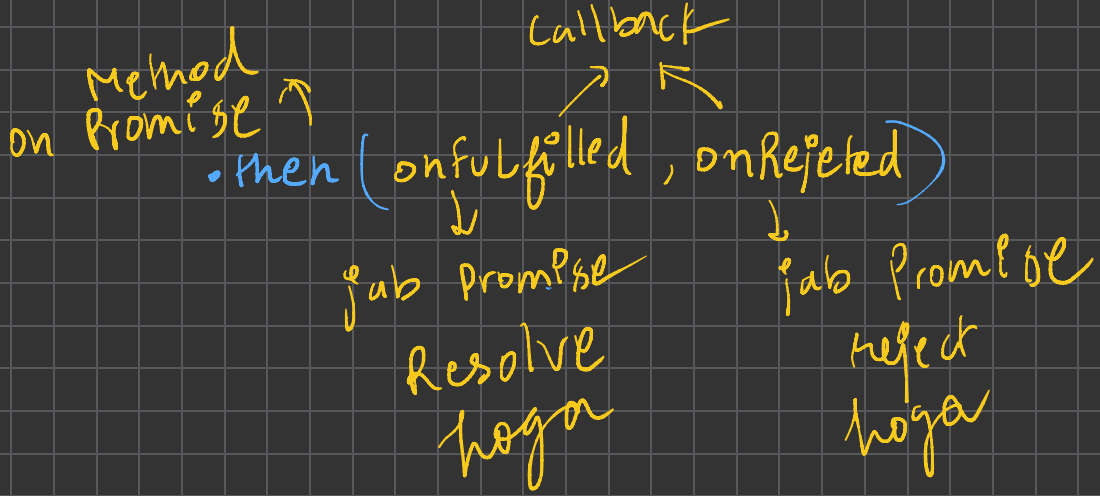
.then() , .catch() , .finally()

all three methods are **asynchronous**

and they uses **microtask queue**

and all three method always returns

again a **promise object**


Method on Promise
.then
onFulfilled
onRejected
callback
jab Promise Resolve hoga
jab Promise reject hoga


const p = new Promise(function (resolve, reject) {
// resolve("hiii")
reject("Server down hai beta")
})
// console.log("c");
// console.log(typeof p); // object
console.log(p);
p.then(function onFulfilled(val) {
console.log(val);
}, function onRejected(val) {
console.log(val);
})

• .then(cb1, cb2)

• .catch()

↳ always for Reject

• .finally() → always runs

either promise fulfilled

or Reject

```
console.log("a")

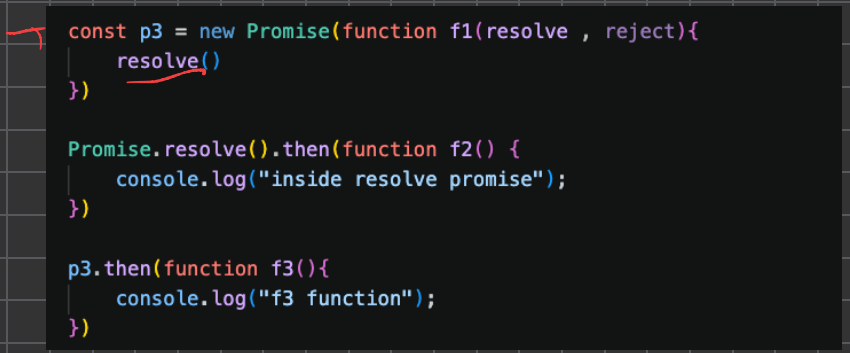
const p2 = new Promise(function f1(resolve , reject) {
    console.log("b");
    resolve("hello")
})

p2.then(function f2(val){
    console.log("then");
    console.log(val);
}).catch(function f3(){
    console.log("catch");
}).finally(function f4(){
    console.log("finally");
})

console.log("c");
```

output

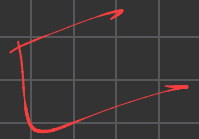
1. a
2. b
3. promise fulfilled with "hello"
4. c
5. f2 → Microtask
6. f2 (inside call stack)
7. then
8. val
9. finally
10. finally (inside call stack)



```
const p3 = new Promise(function f1(resolve , reject){
    resolve()
})

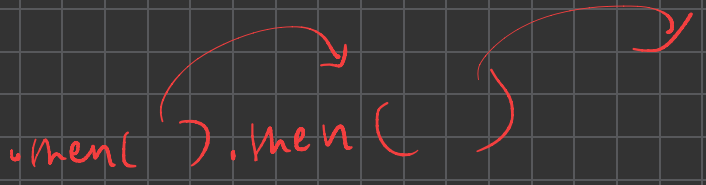
Promise.resolve().then(function f2() {
    console.log("inside resolve promise");
})

p3.then(function f3(){
    console.log("f3 function");
})
```



event loop

callback queue > micro task > macro task


.then( ).then( )

```
function searchPizza() {
    return new Promise(function (resolve, reject) {
        console.log("Pizza searching...");
        setTimeout(function fun1() {
            console.log("Here is the Pizza's Menu.");
            let price = 500;
            // a(price)
            resolve(price)
        }, 2000)
    })

}

function addToCart(price) {
    return new Promise(function (resolve, reject) {
        console.log("Pizza adding to cart...");
        setTimeout(function fun2() {
            console.log("Pizza Added to cart");
            resolve(price)
        }, 3000)
    })
}

function paymet(price) {
    return new Promise(function (resolve, reject) {
        console.log(`Payment Initiated , Amount : ${price}`);
        setTimeout(function fun3() {

            let isPaymentSuccessful = false

            if (isPaymentSuccessful) {
                console.log(`Payment Completed, Amount : ${price}`);
                resolve()
            } else {
                reject("Bhaiya payment failed")
            }


        }, 5000)
    })
}
```

```
let res = searchPizza()

res.then(function (price) {
    return addToCart(price)
}).then(function (price) {
    return paymet(price)
}).then(function () {
    console.log("Bss Aa hee gaya Pizza");
}).catch(function (err) {
    console.log(err);
})
```

promise object
{
state : fulfilled / Reject
result :
}